P.A. COLLEGE OF ENGINEERING,

MANGALORE



Report

On

Subject Based Mini Project

Of

DEEP LEARNING & REINFORCEMENT LEARNING

By The Department Of

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Image Generation Using Generative Adversarial Networks (GANs)

Submitted By

MUHAMMED ISMAIL MAAIZ **4PA23AI041**

Submitted To

Dr. Mohammed Zakir Bellary

1. Introduction

Creating new, realistic images has traditionally required artists, photographers or designers. Classical image editing tools can only modify images that already exist, and they cannot produce entirely new pictures that look like they belong to a real collection.

Generative Adversarial Networks (GANs), introduced by Goodfellow et al. in 2014, solve this problem with a simple idea: two neural networks are trained against each other. The **Generator** takes random noise and tries to produce a realistic image. The **Discriminator** looks at an image and decides whether it is real (from the dataset) or fake (made by the Generator). The Generator improves by trying to fool the Discriminator, and the Discriminator improves by catching the fakes. This competition is called *adversarial training*.

In this project, a Deep Convolutional GAN (DCGAN), proposed by Radford et al. (2016), was implemented in TensorFlow/Keras and trained on the Fashion-MNIST dataset to generate new synthetic images of clothing items. The trained Generator was connected to an interactive interface built with ipywidgets in a Google Colab notebook, where the user can choose a saved training checkpoint, select the number of images, and generate new images with a button click.

2. Objectives

The main objective of this project is to train a neural network that generates new synthetic fashion images from random noise. The specific objectives are:

1. To design a DCGAN Generator that converts a 100-dimensional random noise vector into a 28×28 grayscale image using Dense, Reshape and transposed convolution (Conv2DTranspose) layers.
2. To design a convolutional Discriminator that classifies images as real or fake.
3. To implement a custom adversarial training loop using `tf.GradientTape`, binary cross-entropy loss and separate Adam optimizers for the two networks.
4. To record the training losses and save generated image grids and Generator checkpoints during training.
5. To build a simple interactive interface that uses the trained Generator.
6. To save the trained models in the native Keras format and reload the Generator to generate images without retraining.
7. To study the quality and limitations of the generated images.

3. System Architecture

The system has four main parts: the data pipeline, the Generator, the Discriminator and the interactive interface. During training, the Generator creates fake images from random noise, and the Discriminator receives real images from the dataset as well as these fake images. The Discriminator output is used to compute two losses, one for each network. The gradients of these losses are used by two separate Adam optimizers to update the weights of the Generator and the Discriminator. The overall flow is shown in Fig. 1.

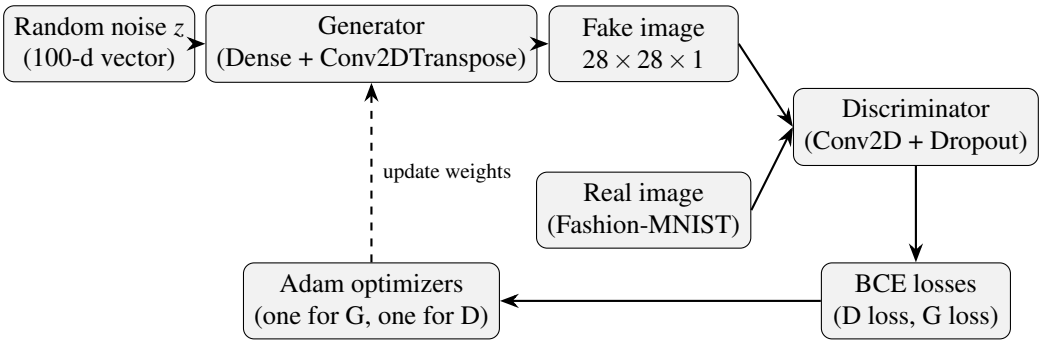


Figure 1: System architecture of the DCGAN training process

Generator. The 100-dimensional noise vector is passed through a Dense layer with $7 \times 7 \times 128$ units and reshaped into a $7 \times 7 \times 128$ feature map. Two transposed convolution layers with stride 2 then increase the size from 7×7 to 14×14 and then to 28×28. A final transposed convolution layer with a Tanh activation produces a $28 \times 28 \times 1$ image with pixel values in $[-1, 1]$. Batch normalization and LeakyReLU are used in the hidden layers to stabilize training (Table 1).

Discriminator. The Discriminator takes a $28 \times 28 \times 1$ image and uses two strided convolution layers to reduce it to 14×14 and then 7×7 feature maps. LeakyReLU activations and Dropout (0.3) are used after each convolution. The result is flattened and passed to a single Dense unit that outputs a logit (a raw score) for real or fake. No Sigmoid is used in the model, because the loss function is configured with `from_logits=True` (Table 2).

Table 1: Generator architecture

Layer	**Details**	**Output shape**
Input	Noise vector z	100
Dense	$7 \times 7 \times 128$ units, no bias	6272
BatchNorm + LeakyReLU	slope 0.2	6272
Reshape	to feature map	$7 \times 7 \times 128$
Conv2DTranspose	64 filters, k=5, s=2, same, BN, LeakyReLU	$14 \times 14 \times 64$
Conv2DTranspose	32 filters, k=5, s=2, same, BN, LeakyReLU	$28 \times 28 \times 32$
Conv2DTranspose	1 filter, k=5, s=1, same, Tanh	$28 \times 28 \times 1$

Table 2: Discriminator architecture

Layer	Details	Output shape
Input	Real or fake image	$28 \times 28 \times 1$
Conv2D	64 filters, k=5, s=2, same, LeakyReLU(0.2), Dropout(0.3)	$14 \times 14 \times 64$
Conv2D	128 filters, k=5, s=2, same, LeakyReLU(0.2), Dropout(0.3)	$7 \times 7 \times 128$
Flatten	–	6272
Dense	1 unit (logit, no activation)	1

Interactive interface. The interface, built with ipywidgets, loads the saved Generator checkpoints. The user selects a checkpoint (epoch) from a dropdown, sets the number of images with a slider, and clicks *Generate new images*. A second button shows the training loss graph, and a status message tells the user what the application is doing. A Matplotlib-based fallback function is provided in case ipywidgets does not work.

4. Dataset Used

The Fashion-MNIST dataset, released by Zalando Research (Xiao, Rasul and Vollgraf, 2017), was used. It contains 60,000 training images and 10,000 test images of clothing items. Each image is a 28×28 grayscale picture, and there are 10 classes: T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag and ankle boot. The dataset is downloaded automatically through Keras, so no manual download is needed. It is small and light, which makes it suitable for training a GAN quickly in a student environment.

A GAN learns the overall distribution of the images, so the class labels and the test set were not used. Only the 60,000 training images were used.

Preprocessing. The pixel values were converted from the range $[0, 255]$ to $[-1, 1]$ using $x' = (x - 127.5)/127.5$. This matches the Tanh output of the Generator, so real and generated images lie on the same scale. A channel dimension was added so that each image has the shape $28 \times 28 \times 1$. The data was then shuffled and divided into batches of 128 images using `tf.data`.



Figure 2: Sample real images from the Fashion-MNIST dataset

5. Technologies Used

Table 3: Technologies used

Technology	Purpose
Python	Core programming language
TensorFlow / Keras	Deep learning framework (models, GradientTape, Adam optimizer, dataset loading)
NumPy	Array handling and image scaling
Matplotlib	Displaying images, image grids and loss curves
ipywidgets	Interactive buttons, slider and dropdown in the notebook
Google Colab	Development and training environment (GPU when available)

6. Methodology

Setup. The required libraries were imported, the availability of a GPU was checked (training also runs on a CPU, but more slowly), and the random seeds of Python, NumPy and TensorFlow were fixed to 42 for reproducible results.

Data preparation. The dataset was loaded, normalized to $[-1,1]$ and batched as described in Section 4.

Loss functions. The Discriminator is trained to output 1 for real images and 0 for fake images. With binary cross-entropy (BCE), its loss is

$$L_D = -\mathbb{E}_x\left[\log D(x)\right] - \mathbb{E}_z\left[\log\left(1 - D(G(z))\right)\right].$$

The Generator is trained to make the Discriminator classify its images as real. The commonly used non-saturating form of its loss is

$$L_G = -\mathbb{E}_z\left[\log D(G(z))\right],$$

which gives stronger gradients to the Generator early in training. In the code, BCE is used with `from_logits=True`, so the Sigmoid is applied inside the loss function, which is numerically more stable.

Training loop. A custom training step, compiled with `tf.function`, was written using `tf.GradientTape`. In each step, a batch of noise vectors is passed through the Generator to create fake images. The Discriminator then scores the real batch and the fake batch. The Discriminator loss (real images should score 1, fake images 0) and the Generator loss (fake images should score 1) are computed, and two gradient tapes give the gradients of each loss

with respect to the weights of its own network. Two separate Adam optimizers then update the Generator and the Discriminator.

Monitoring and checkpoints. The average Generator and Discriminator losses were recorded at the end of every epoch. A fixed set of 16 noise vectors was used to generate and save an image grid after every epoch, so the progress can be compared on the same inputs. The Generator was also saved as a checkpoint after every epoch, which allows the interface to generate images from any saved stage of training.

Table 4: Training configuration

Parameter	**Value**
Model	DCGAN (Generator and Discriminator)
Dataset	Fashion-MNIST (60,000 training images)
Image size	28×28 pixels, 1 channel
Noise dimension	100
Batch size	128
Epochs	10
Optimizer	Adam (separate for G and D), learning rate 2×10^{-4}, $\beta_1 = 0.5$
Loss	Binary cross-entropy (`from_logits=True`)
Random seed	42
Hardware	Google Colab (GPU when available)

Saving and reloading. After training, the Generator and the Discriminator were saved in the native Keras format (`.keras`). The Generator can be reloaded with `keras.models.load_model` and used to create new images without any retraining.

7. Evaluation Metrics

A GAN has no single correct output, and its loss values do not behave like the loss of a classifier, so accuracy cannot be used. The system was evaluated in the following ways.

- **Loss curves:** the Generator and Discriminator losses were plotted over all epochs. Curves that stay in a similar range, without one network completely dominating, indicate that training is balanced.
- **Visual inspection:** image grids from different epochs were checked to see whether clothing shapes (shirts, shoes, bags and so on) become recognizable and whether the images contain noise or broken regions.
- **Diversity check:** the grids were checked for repeated images or a single dominant category, which would indicate mode collapse.

No quantitative image-quality metric such as the Fréchet Inception Distance (FID) was computed, so the quality was judged visually only.

8. Results

Fig. 3 shows images generated from the same fixed noise vectors at different epochs. In the first epoch the images are only blurry blobs with little structure. As training continues, the Generator learns the outlines of the clothing items and the different categories become recognizable.

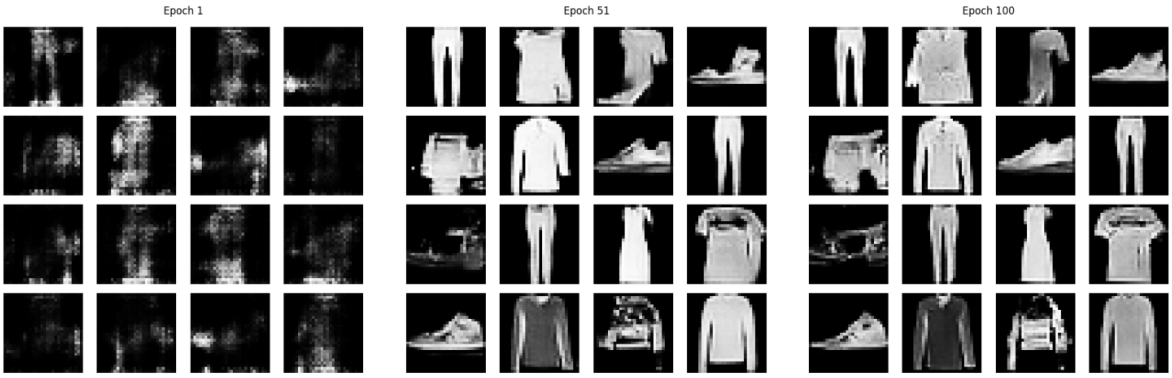


Figure 3: Generated images at the first, middle and last epoch (same fixed noise)

The loss curves of the two networks are shown in Fig. 4.

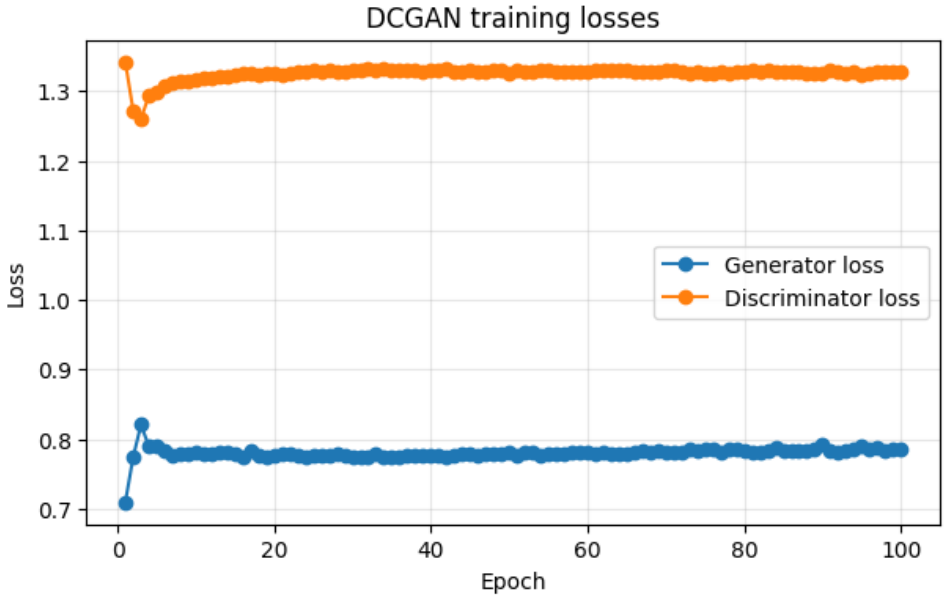


Figure 4: Generator and Discriminator loss during training

Fig. 5 shows 16 new images generated by the reloaded Generator from fresh random noise that was not used during training.

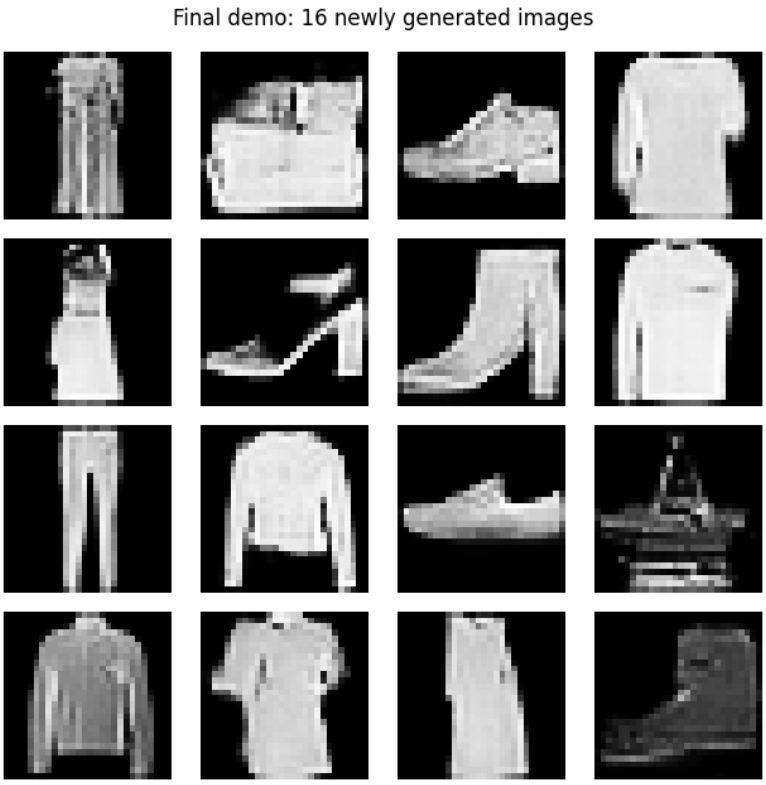


Figure 5: Final demonstration: 16 newly generated images from random noise

The interactive interface is shown in Fig. 6. The user selects the checkpoint and the number of images, and the generated images are displayed below the controls together with a status message.

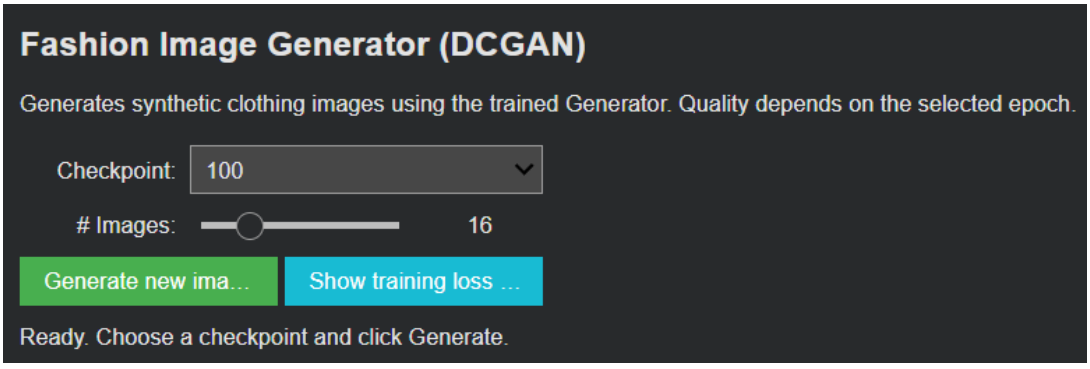


Figure 6: Interactive interface for generating images from saved checkpoints

Discussion. The results show that the Generator learned the general shapes of clothing items from the dataset without using any labels. Because every checkpoint is saved, the interface also lets the user compare the output of early and late stages of training.

Some limitations can also be seen. The model was trained for only 10 epochs, so the images are recognizable but not realistic, and some images contain noisy textures or unclear shapes. The Generator may also favour a few easy categories, such as shoes or bags, and repeat similar

images, which is called mode collapse. GAN training is unstable in general, and the loss values alone do not show how good the images are. Finally, quality was judged visually only, and the user cannot choose which type of clothing is generated.

9. Conclusion

This project demonstrates the use of a Deep Convolutional GAN for image generation. A Generator and a Discriminator were built in TensorFlow/Keras and trained against each other on the Fashion-MNIST dataset, using binary cross-entropy loss, a custom `GradientTape` training loop and separate Adam optimizers. The generated images improved during training, starting from blurry blobs and developing recognizable clothing shapes. The trained Generator was saved and reloaded to create new images without retraining, and an interactive interface was built in which the user can choose a checkpoint and generate new images.

The main limitations are the short training time, the small image size, the lack of control over the clothing type and the possibility of mode collapse. Despite these limits, the project shows how two competing networks can learn to create new images from random noise without any labels.

10. Future Scope

The project can be improved in several ways. Training for more epochs can produce sharper and more varied images. A conditional GAN that uses the 10 class labels would let the user choose the type of clothing to generate. More stable training methods such as the Wasserstein GAN with gradient penalty (WGAN-GP) can reduce instability and mode collapse. A quantitative metric such as the Fréchet Inception Distance (FID) can be added to measure image quality. The same model can also be applied to larger and more complex datasets at a higher resolution. Finally, the application can be deployed as a web application so that it can be used from any device.

11. References

1. I. Goodfellow et al., “Generative Adversarial Nets,” Advances in Neural Information Processing Systems (NeurIPS), 2014.

2. A. Radford, L. Metz and S. Chintala, “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks,” ICLR, 2016 (arXiv:1511.06434).

3. H. Xiao, K. Rasul and R. Vollgraf, “Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms,” arXiv:1708.07747, 2017.

4. D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” ICLR, 2015.

5. TensorFlow and Keras Documentation, https://www.tensorflow.org/api_docs

6. ipywidgets Documentation, https://ipywidgets.readthedocs.io